

EX-10 Write the Python program to implement A*Algorithm

AIM

To develop a Python program that implements the A* (A-Star) Search Algorithm to find the shortest path between a start node and a goal node using heuristic values.

ALGORITHM

Step 1: Start the program.

Step 2: Define the graph with nodes and edge costs.

Step 3: Define the heuristic value $h(n)$ for each node.

Step 4: Initialize the open list with the start node and an empty closed list.

Step 5: Select the node with the lowest $f(n) = g(n) + h(n)$ from the open list.

Step 6: If the selected node is the goal node, display the shortest path and stop.

Step 7: Otherwise, expand all its neighboring nodes.

Step 8: Update the path cost (g), heuristic (h), and total cost (f) for each neighbor.

Step 9: Repeat Steps 5–8 until the goal node is reached or no path exists.

Step 10: Stop the program.

PROGRAM CODE

```
graph = {  
    'A': [('B', 1), ('C', 3)],  
    'B': [('D', 3), ('E', 6)],  
    'C': [('F', 5)],  
    'D': [('G', 2)],  
    'E': [('G', 1)],  
    'F': [('G', 2)],  
    'G': []
```

```
}
```

```
heuristic = {
```

```
    'A': 7,
```

```
    'B': 6,
```

```
    'C': 4,
```

```
    'D': 2,
```

```
    'E': 1,
```

```
    'F': 2,
```

```
    'G': 0
```

```
}
```

```
def a_star(start, goal):
```

```
    open_list = [(start, 0)]
```

```
    g_cost = {start: 0}
```

```
    parent = {start: None}
```

```
    while open_list:
```

```
        open_list.sort(key=lambda x: g_cost[x[0]] + heuristic[x[0]])
```

```
        current = open_list.pop(0)[0]
```

```
        if current == goal:
```

```
            path = []
```

```
            while current:
```

```

        path.append(current)

        current = parent[current]

    print("Shortest Path:", " -> ".join(path[::-1]))

    return

for neighbor, cost in graph[current]:

    new_cost = g_cost[current] + cost

    if neighbor not in g_cost or new_cost < g_cost[neighbor]:

        g_cost[neighbor] = new_cost

        parent[neighbor] = current

        open_list.append((neighbor, new_cost))

print("Path not found")

start = input("Enter Start Node: ")
goal = input("Enter Goal Node: ")

a_star(start, goal)

```

OUTPUT

```

=====
Enter Start Node: A
Enter Goal Node: G
Shortest Path: A -> B -> D -> G

```

RESULT

The Python program successfully implements the **A* (A-Star) Search Algorithm** to find the shortest path from the start node to the goal node using heuristic values. It efficiently selects the optimal path by minimizing the total estimated cost.